



UNIVERSITY
of
INFORMATION TECHNOLOGY

CST - 8114

Distributed Programming

Project Report

Ace Play:Card Game

Group Members

Phyo Than Thar Kyaw [4SE-1016]

Su Htet Thinzar[4SE-1094]

Hein Htet Kyaw [4SE-1101]

Sithu Aung [4SE-1114]

Thiri Shwe Sin [4SE-1145]

[SE-Group 2]

Submitted Date: 10.9.2024

Table of Contents

Abstract	
1. Introduction	
1.1. Introduction	
1.2. Objectives of the System	
1.3. Requirements	
2. Project Specification	
2.1 Scope of this project	
3. Project System Architecture	
3.1 System Flow Diagram	
3.2 User Interface	
4. Testing	
4.1. Functional Testing	
4.2. Usability Testing	
4.3 Reliability Testing	
4.4 Availability Testing	
5. Application Configuration	
5.1. Software Requirements.	
5.2. Hardware Requirements	
6. Conclusion	
6.1.Limitations	
6.2.Future Improvements	
7. References	

Abstract

This project presents the design and implementation of a Python-based card game simulation incorporating multiplayer functionality using a client-server architecture. The game follows custom rules where each player starts with two cards and can draw one additional card. The total score of a hand is determined by summing the card values, and if the total exceeds 10, only the last digit is used (e.g., 15 = 5). The player with the highest score wins. In cases where the scores are tied, tiebreakers are applied based on the number of cards held, the highest card rank, and, if necessary, by suit order (Spade, Heart, Diamond, Club).

To provide an interactive experience, a graphical user interface (GUI) was developed using the Tkinter library in Python. This GUI allows players to visualize their cards, monitor their score, and interact with the game seamlessly. The project also utilizes MySQL as the database management system to store essential data such as player information, game results, and historical statistics, supporting future expansion for more complex data management needs.

The network functionality is powered by Socket programming, enabling real-time multiplayer gameplay. A client-server model was implemented with the server managing game logic, player connections, and game flow, while clients handle user input and GUI updates. This architecture ensures smooth synchronization and real-time updates between players. Extensive testing demonstrated the system's reliability in handling tiebreaker rules and real-time communication.

Future development could focus on optimizing network performance for larger player groups, enhancing the GUI for a more engaging experience, and introducing AI opponents for single-player modes. Additional enhancements might involve expanding the MySQL database for more detailed statistics and implementing advanced game modes.

In conclusion, this Python card game project effectively integrates GUI design, database management, and network communication to create a multiplayer gaming experience. It highlights the practical application of these concepts and provides a foundation for future enhancements and more complex game development.

1. Introduction

1.1. Introduction

The card game industry has long been a source of entertainment, offering players both fun and challenge through strategic gameplay. With the evolution of technology, many traditional card games have transitioned into digital formats, providing the opportunity for players to enjoy their favorite games virtually and even compete with others in real-time. This project aims to simulate a multiplayer card game using Python, combining various programming techniques such as graphical user interface (GUI) design, database management, and network communication.

The primary focus of this project is to implement a custom card game where each player is initially dealt two cards and has the option to draw a third. The game's unique scoring system adds an interesting twist, as only the last digit of the total score is considered if the combined value of the cards exceeds 10. To determine the winner, the game applies a series of tiebreaker rules based on the number of cards held, card rank, and suit hierarchy.

This report outlines the design, development, and functionality of the card game project. It covers the game's rules, system architecture, database structure, and network design, along with an overview of its performance during testing. Furthermore, this project serves as a learning experience for integrating multiple programming concepts into a cohesive and fully operational multiplayer game, with room for future enhancements.

1.2. Objectives of the System

The primary objective of this project is to develop a Python-based multiplayer card game that allows players to compete in real-time using a client-server architecture. The system is designed to handle essential game mechanics, such as distributing cards, enforcing rules, and determining winners. A key aspect of the game is its

unique scoring system, where the combined value of the cards is used, but if the total exceeds 10, only the last digit is considered. The system also incorporates a set of tiebreaker rules that take into account the number of cards held, the rank of the highest card, and the suit hierarchy, ensuring a fair outcome.

Another objective is to provide a user-friendly experience through a graphical user interface (GUI) developed with Tkinter. This interface visually represents each player's hand, score, and game progress, making the game easy to follow and interact with. To ensure smooth gameplay, the project leverages Socket programming to facilitate real-time communication between clients and the server, allowing game data to be synchronized across all connected players.

In addition, the system incorporates a MySQL database to manage and store essential game data such as player information, game results, and session history. This ensures that data is persistent across multiple sessions, creating a robust framework for tracking player performance. The system is also designed to handle various game scenarios, including tiebreakers and different win conditions, while maintaining fairness and accuracy in determining the winner.

Finally, the system is built with scalability in mind, allowing for future enhancements. This includes potential updates like adding AI opponents for single-player modes, introducing more complex game modes, and improving the user interface to enhance the overall gaming experience.

1.3. Requirements

Functional Requirements

User Interface:

- ◆ The game must provide a GUI where players can view their cards, scores, and game status.
- ◆ The interface should allow players to interact with the game (e.g., drawing a card, ending their turn).
- ◆ A login page must be presented for each player to sign in before starting the game.

- ◆ After the login, the game rules should be displayed to ensure that players understand the rules before gameplay.
- ◆ A message box should prompt players to decide whether to draw a third card or not during their turn.

Game Logic:

- ◆ The system must correctly implement the card dealing, scoring rules, and tiebreaker mechanisms.
- ◆ The game should handle edge cases like ties and identical cards/suits according to the defined rules.

Multiplayer Support:

- ◆ The system must allow multiple clients to connect and play against each other via a central server.
- ◆ Real-time communication between clients and server should ensure smooth and synchronized gameplay.

Database Management:

- ◆ Player data, including login credentials, game results, and session history, should be stored in the MySQL database.
- ◆ The database must support data persistence across multiple sessions.

Real-Time Communication:

- ◆ The server should handle incoming client requests, manage game states, and broadcast updates to all connected clients.
- ◆ Clients must be able to send commands to the server and receive updates in real-time.

Non-Functional Requirements

Performance:

- ◆ The system should handle up to 2-5 simultaneous players with minimal latency.
- ◆ The server must respond to client requests in real-time to ensure smooth gameplay.

Scalability:

- ◆ The architecture should be scalable to allow future enhancements, such as adding AI players or increasing the number of players.

Reliability:

- ◆ The system must ensure consistent game outcomes, even under high network traffic or multiple players.

Usability:

- ◆ The user interface should be intuitive, enabling players of varying technical skills to participate easily.

Security:

- ◆ Player data and game sessions must be protected to prevent unauthorized access or tampering.

2. Project Specification

2.1. Scope of this project

The scope of this project is to develop a Python-based multiplayer card game that allows users to compete in real-time using a client-server architecture. The game will be implemented with a graphical user interface (GUI) built using Tkinter, providing players with an interactive and user-friendly experience. The game follows custom rules, where each player is dealt two cards, with an option to draw a third card. The scoring system, tiebreakers, and suit hierarchy are integral to the gameplay, ensuring fairness and competitive balance.

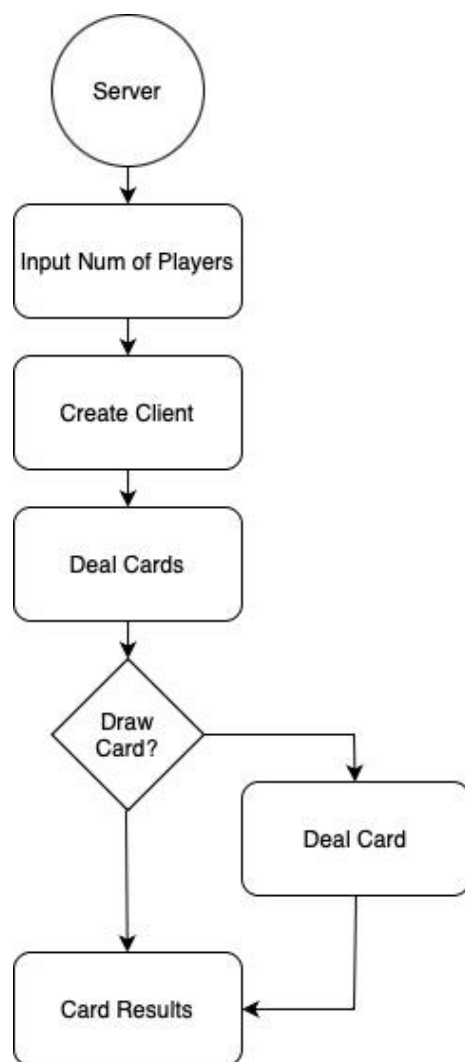
In terms of functionality, the system will support user authentication through a login page, display the game rules after login, and prompt players via a message box to decide whether to draw a third card. MySQL will be used to store player data, game results, and session history, ensuring data persistence across multiple games. The project also employs Socket programming to enable real-time communication between clients and the server, maintaining synchronization and smooth gameplay.

This project is designed to handle up to 2-5 players simultaneously and includes the potential for future enhancements, such as adding AI opponents, expanding the player base, or improving the user interface. However, the current implementation focuses on multiplayer interaction, real-time gameplay, and ensuring the system can manage various game outcomes efficiently.

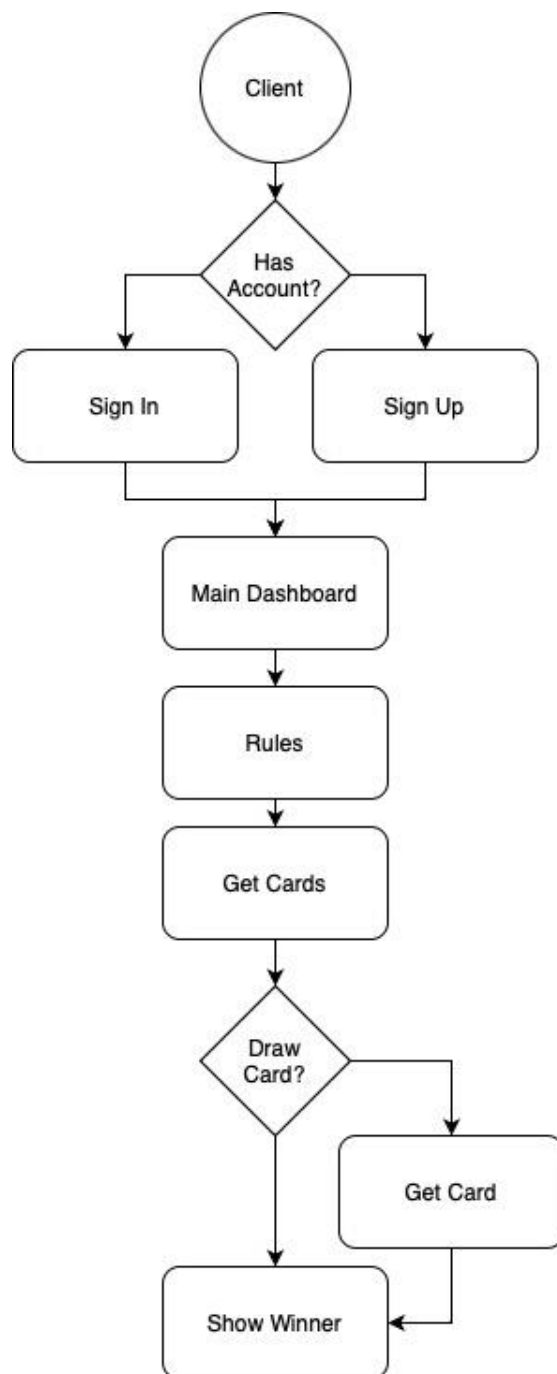
3. Project System Architecture

3.1 System Flow Diagram

Server side



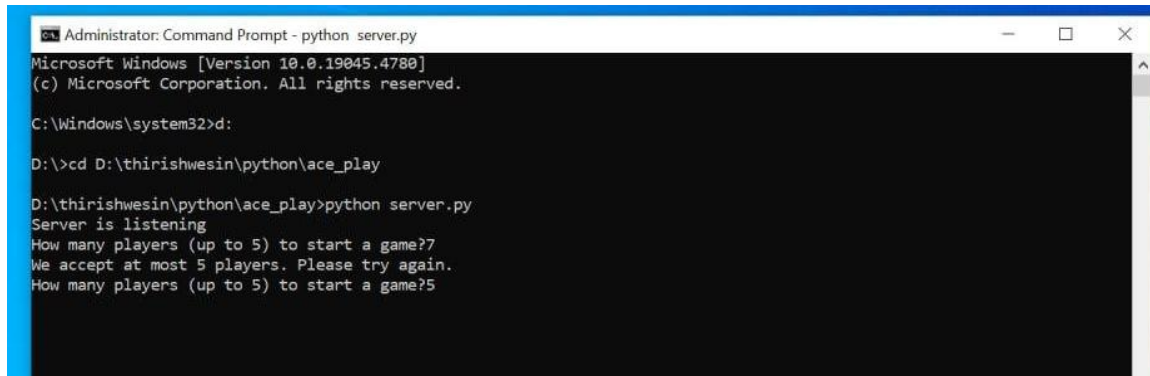
Client side



3.2 User Interface

Server

Start by running the server and then input the number of players to create clients.



```
Administrator: Command Prompt - python server.py
Microsoft Windows [Version 10.0.19045.4780]
(c) Microsoft Corporation. All rights reserved.

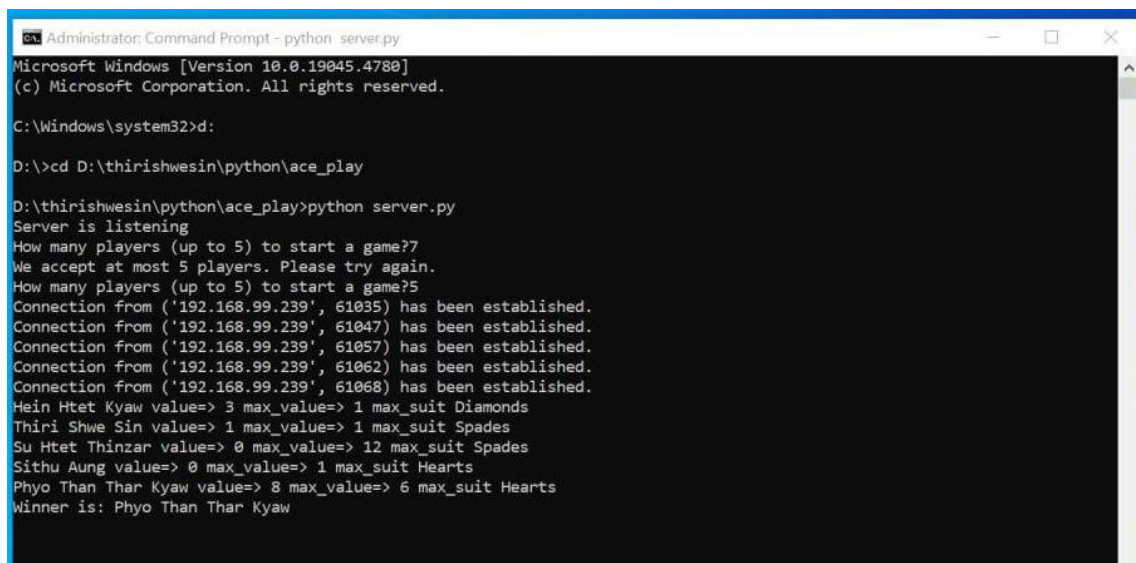
C:\Windows\system32>d:

D:\>cd D:\thirishwesin\python\ace_play

D:\thirishwesin\python\ace_play>python server.py
Server is listening
How many players (up to 5) to start a game?7
We accept at most 5 players. Please try again.
How many players (up to 5) to start a game?5
```

Figure 1.1

Show connections to clients using sockets and card results after the game ends.



```
Administrator: Command Prompt - python server.py
Microsoft Windows [Version 10.0.19045.4780]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>d:

D:\>cd D:\thirishwesin\python\ace_play

D:\thirishwesin\python\ace_play>python server.py
Server is listening
How many players (up to 5) to start a game?7
We accept at most 5 players. Please try again.
How many players (up to 5) to start a game?5
Connection from ('192.168.99.239', 61035) has been established.
Connection from ('192.168.99.239', 61047) has been established.
Connection from ('192.168.99.239', 61057) has been established.
Connection from ('192.168.99.239', 61062) has been established.
Connection from ('192.168.99.239', 61068) has been established.
Hein Htet Kyaw value=> 3 max_value=> 1 max_suit Diamonds
Thiri Shwe Sin value=> 1 max_value=> 1 max_suit Spades
Su Htet Thinzar value=> 0 max_value=> 12 max_suit Spades
Sithu Aung value=> 0 max_value=> 1 max_suit Hearts
Phyo Than Thar Kyaw value=> 8 max_value=> 6 max_suit Hearts
Winner is: Phyo Than Thar Kyaw
```

Figure 1.2

Client

On each client computer, run the file `main_window.py`

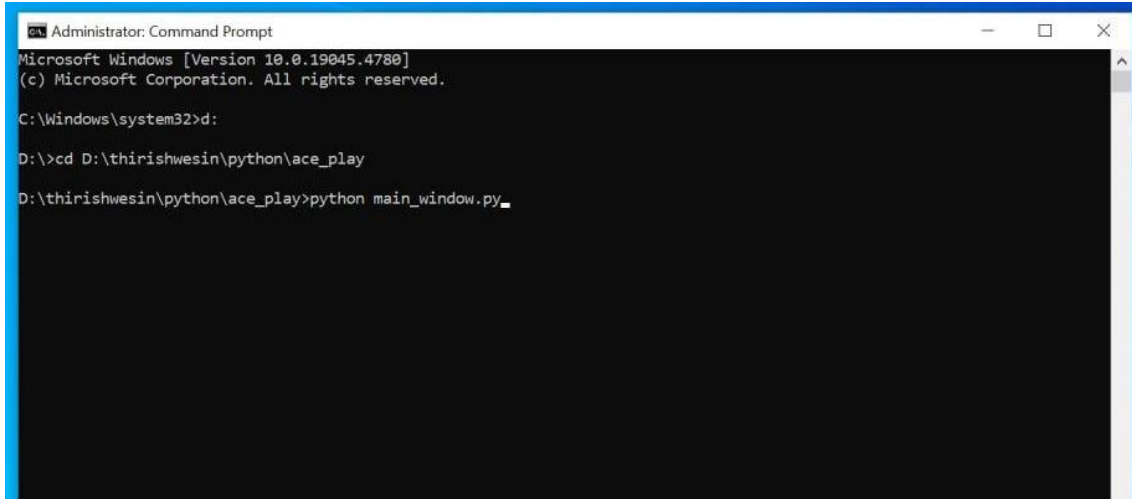


Figure 1.3

A Sign-In menu will appear. Proceed to sign in if you already have an account. If not, go to sign up.

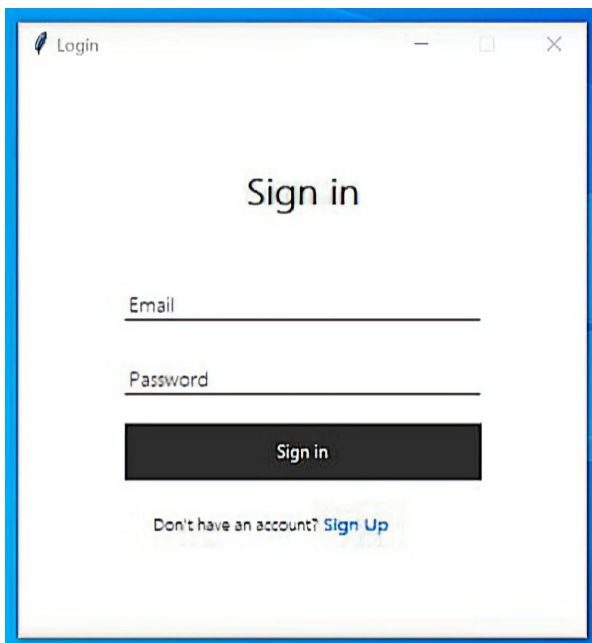


Figure 2.1

This is for users who do not have an account yet. Proceed to fill in the fields.

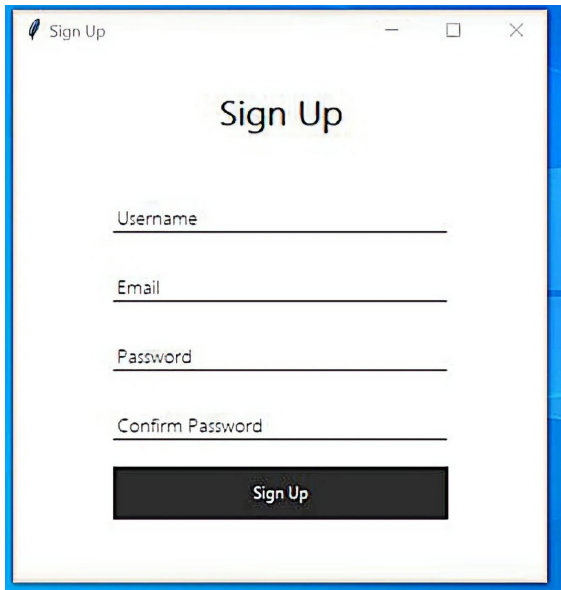
A screenshot of a web application window titled "Sign Up". The window has a white background and a blue border. It contains four text input fields labeled "Username", "Email", "Password", and "Confirm Password" stacked vertically. Below the fields is a dark grey button with the text "Sign Up" in white. The window has standard OS window controls (minimize, maximize, close) in the top right corner.

Figure 2.2

The following are validation checks. This error message to check if the entry fields have been filled or not.

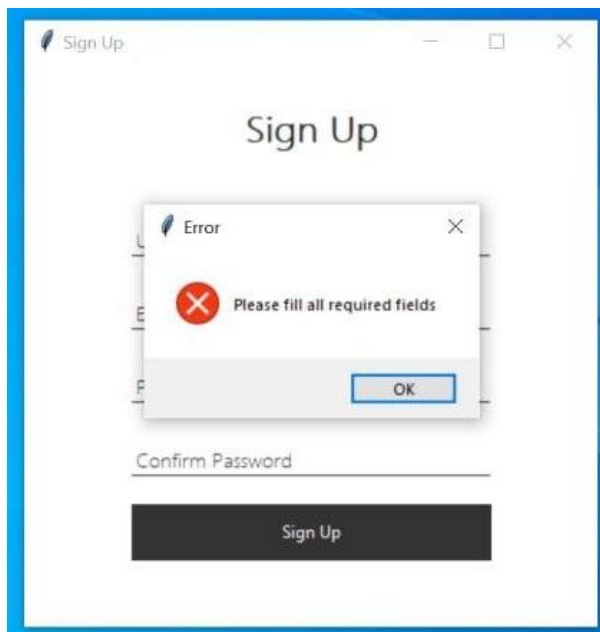


Figure 2.3

This error message will pop up if the input passwords do not match.

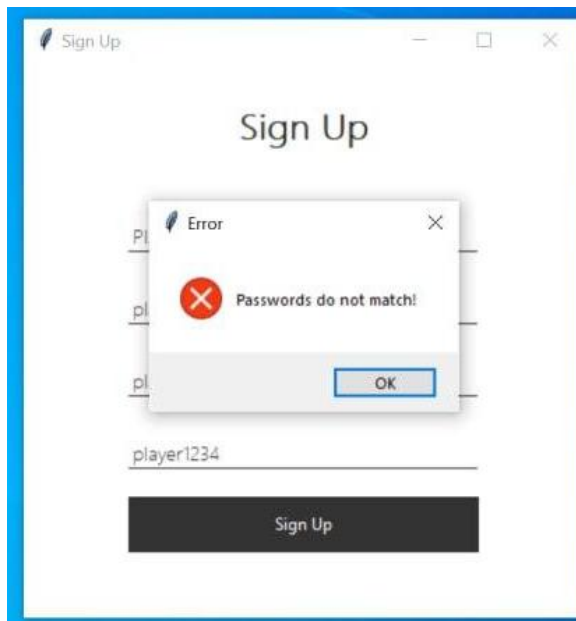


Figure 2.4

This validation is to ensure the users give a strong password.

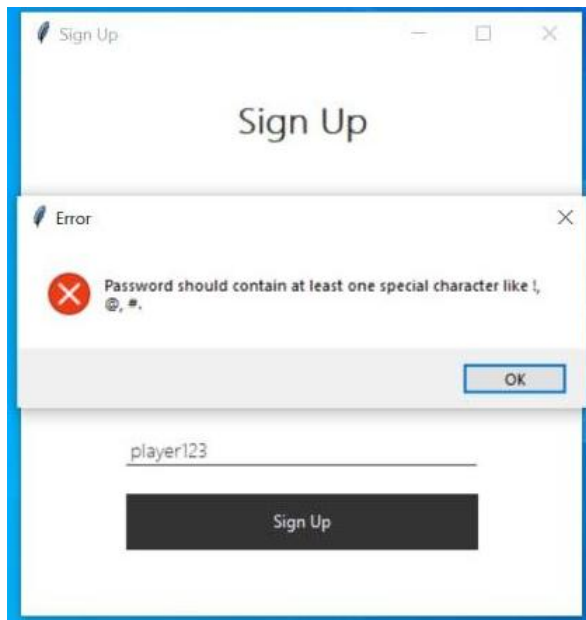


Figure 2.5

After Signing Up, proceed to sign in.

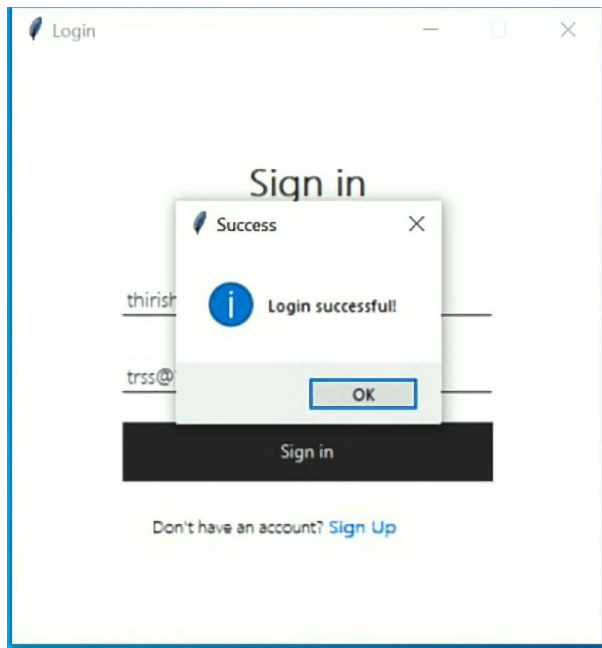


Figure 2.6

After signing in, the game main dashboard will appear. Choose the game mode.



figure 3.1

Game rules displayed after login, explaining the mechanics of card value calculation, scoring, and tie-breaking procedures.

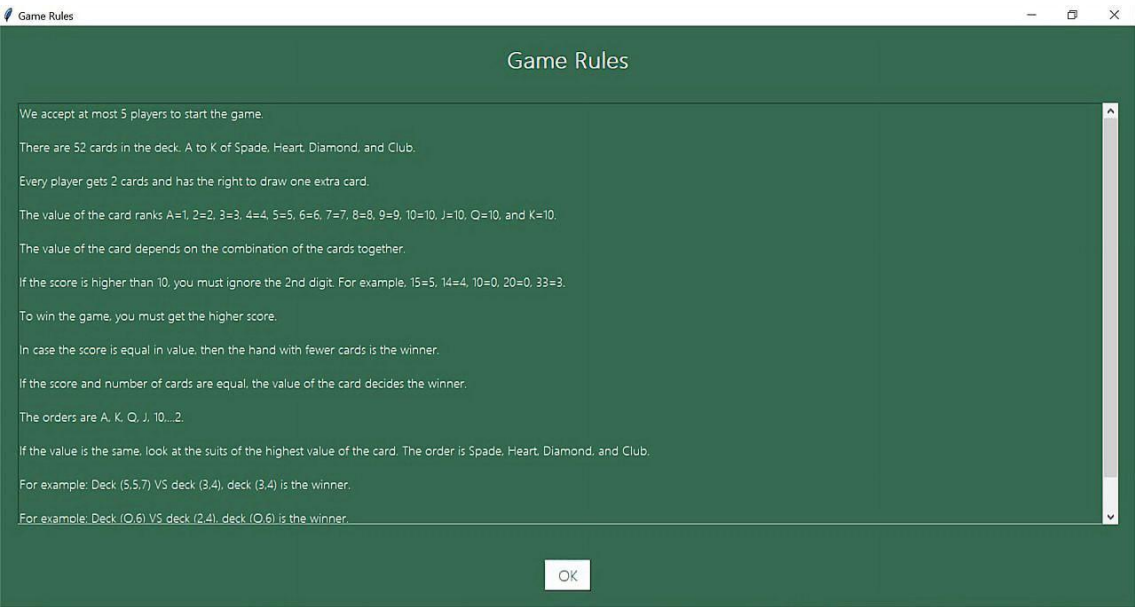


figure 3.2

After reading the rules, wait for all the players to be ready.

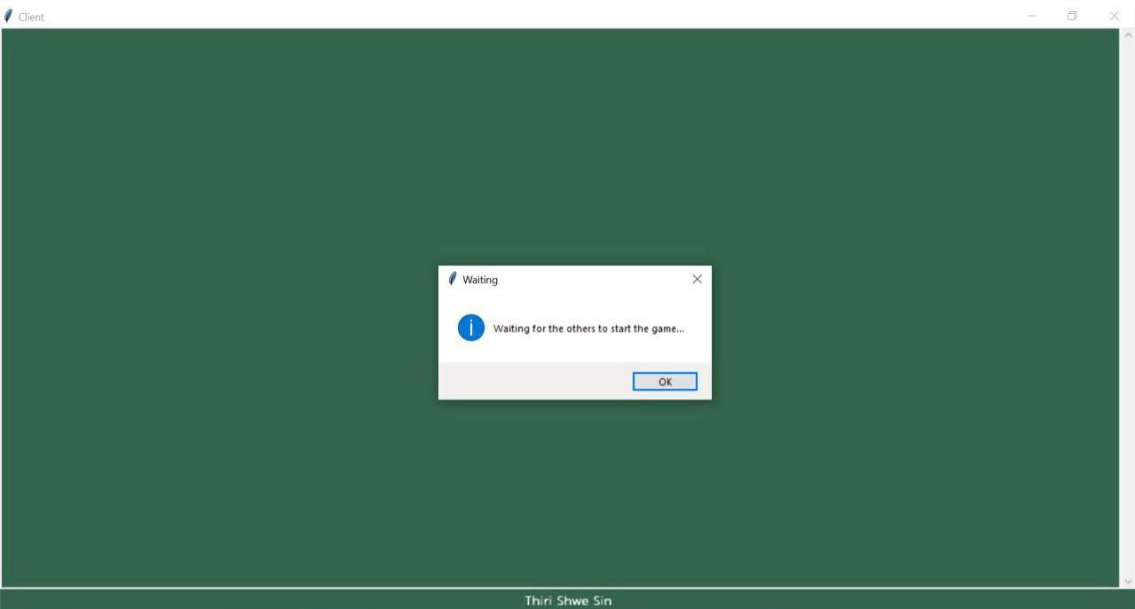


figure 3.3

Two cards are dealt to all players. They can then choose to draw a card or pass.

First Player's Screen

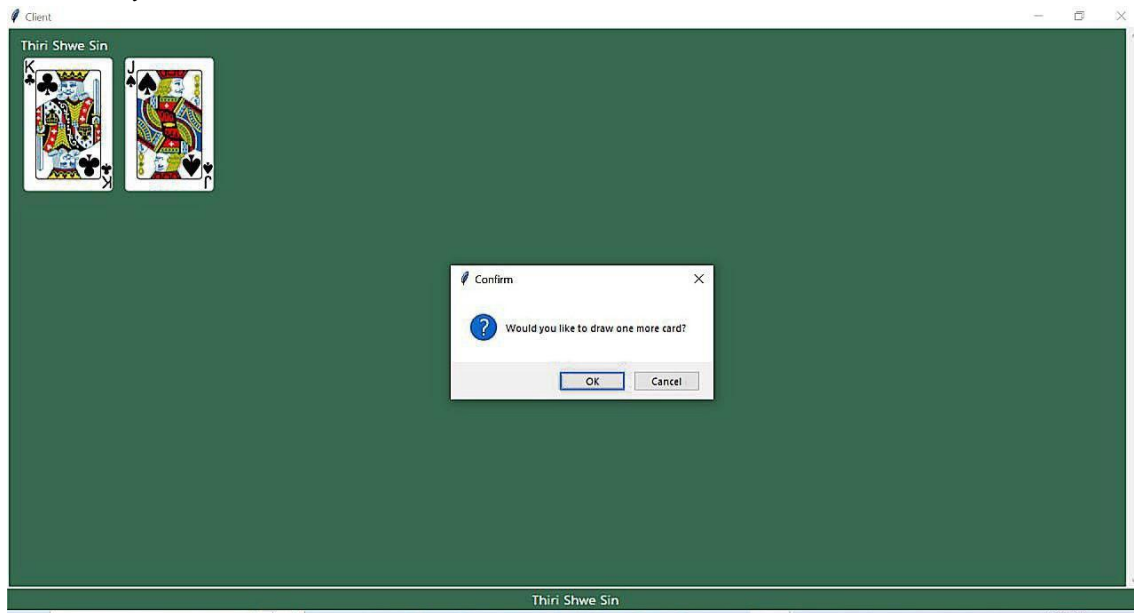


figure 3.4

Second Player's Screen

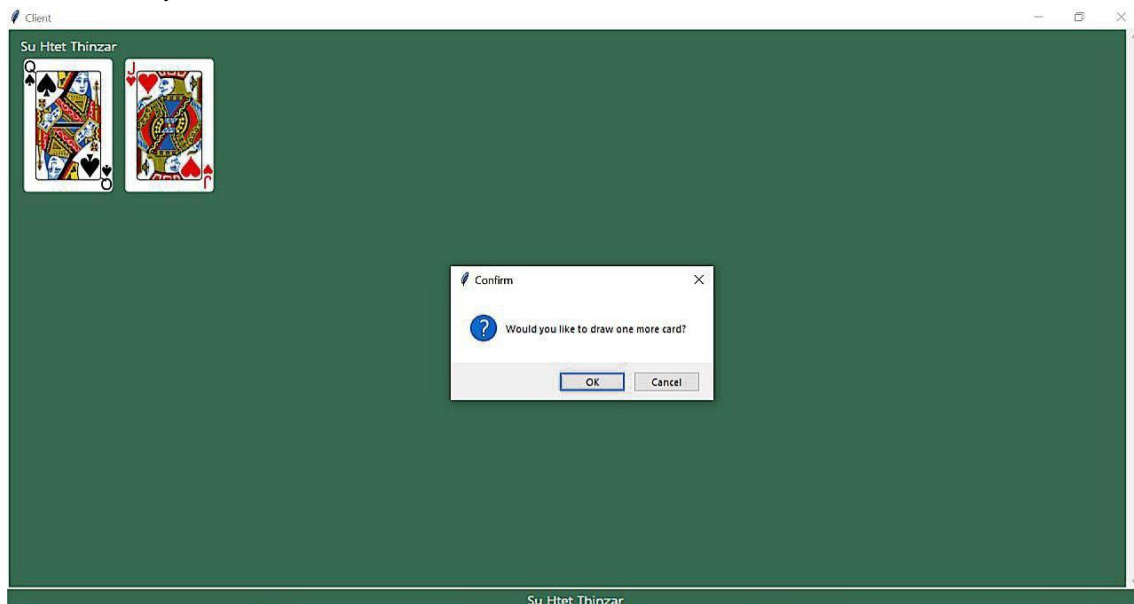


figure 3.5

Third Player's Screen

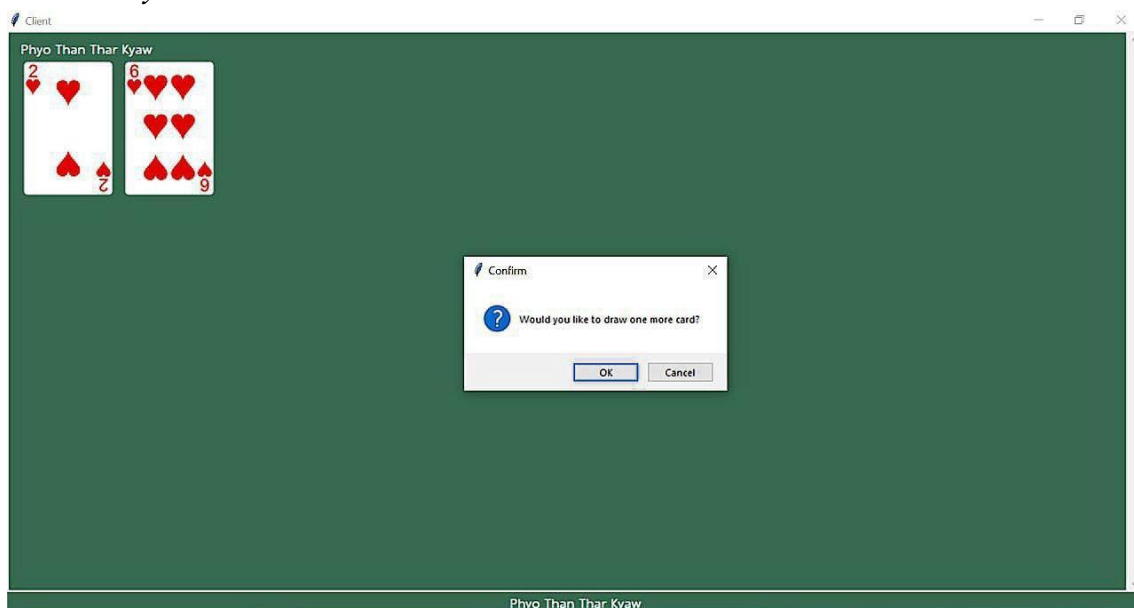


figure 3.6

Fourth Player's Screen

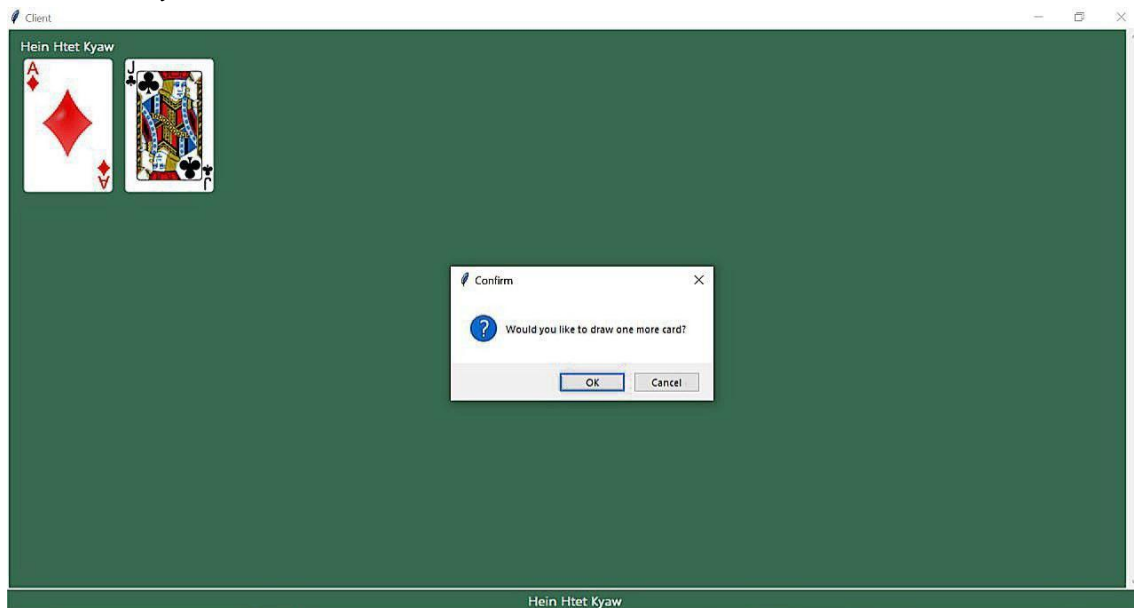


figure 3.7

Fifth Player's Screen

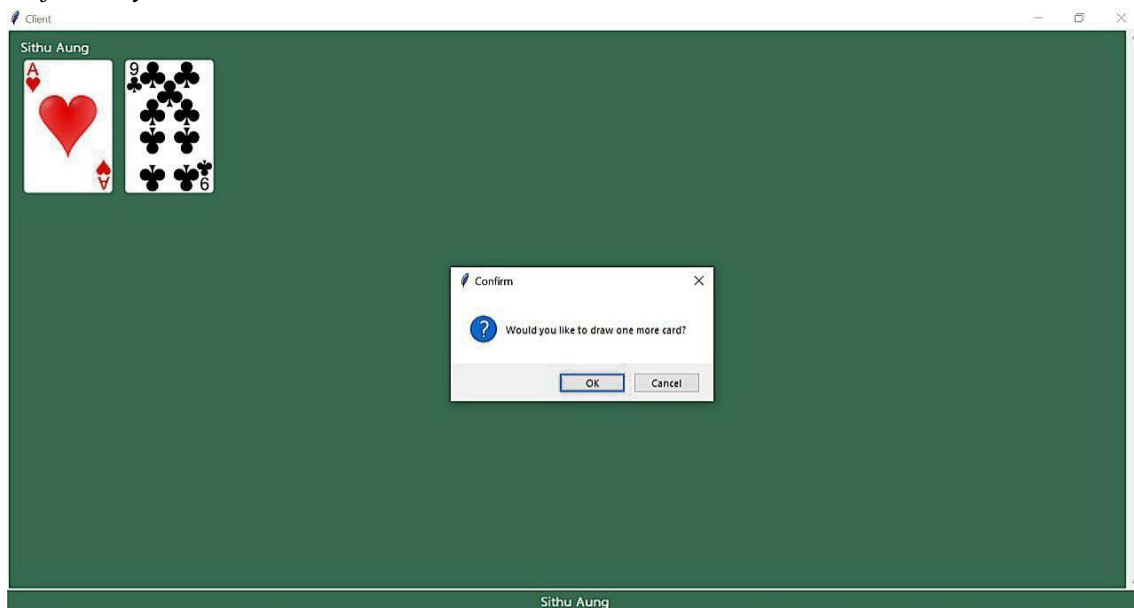


figure 3.8

The final cards will be shown after making the choice. Winner will be shown after all players have made their choices.

Fourth Player's Screen

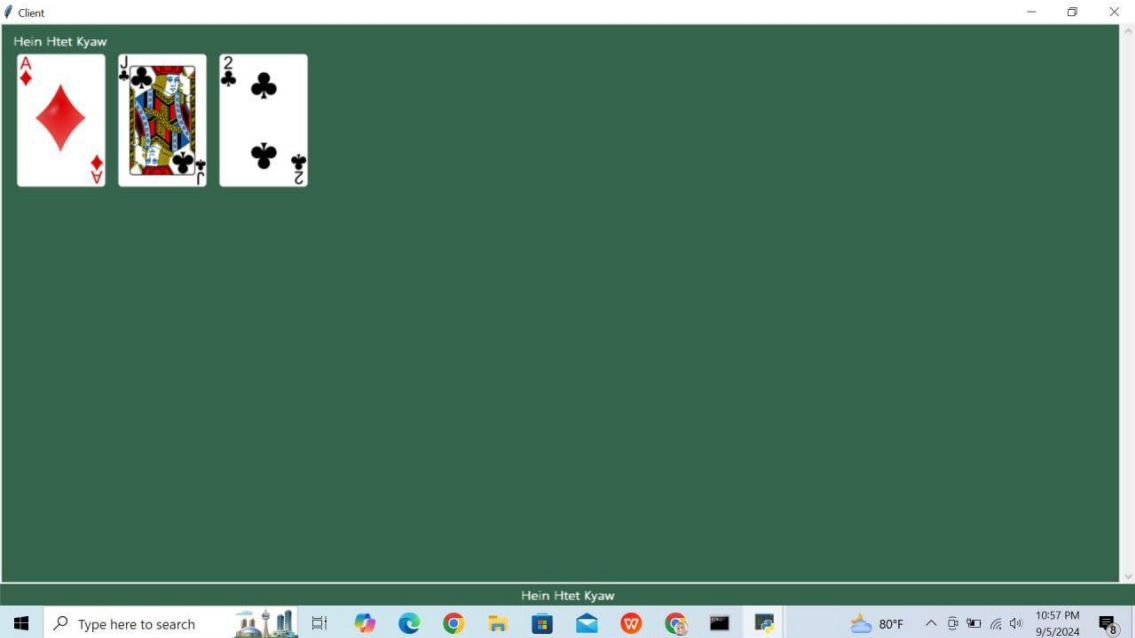


figure 3.9

First Player's Screen

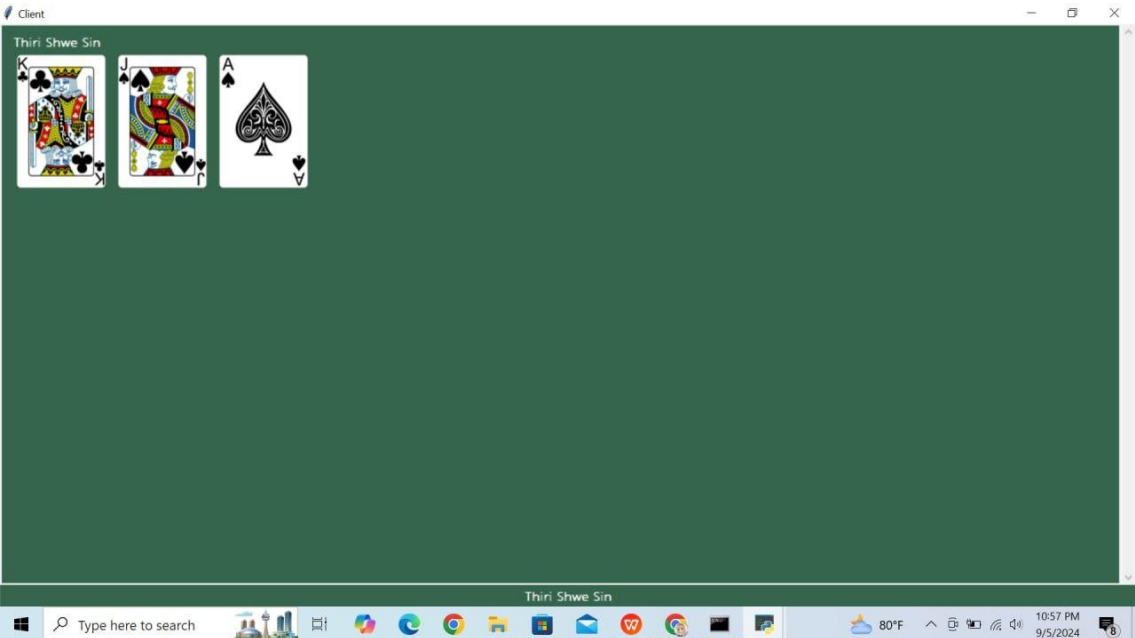


figure 3.10

Fifth Player's Screen

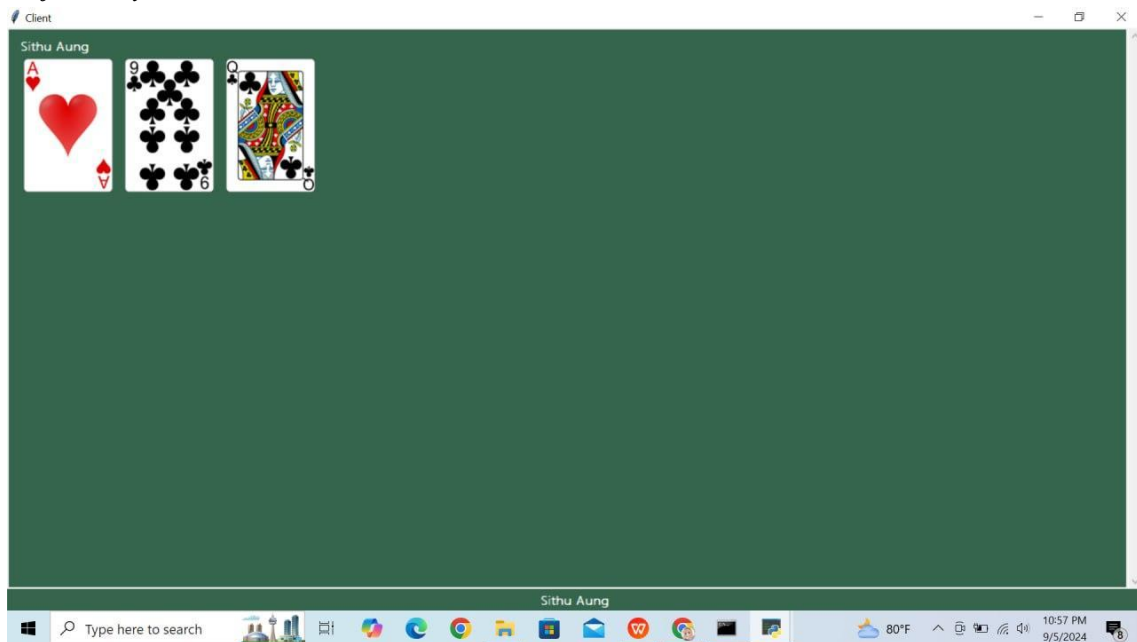


figure 3.11

Third Player's Screen

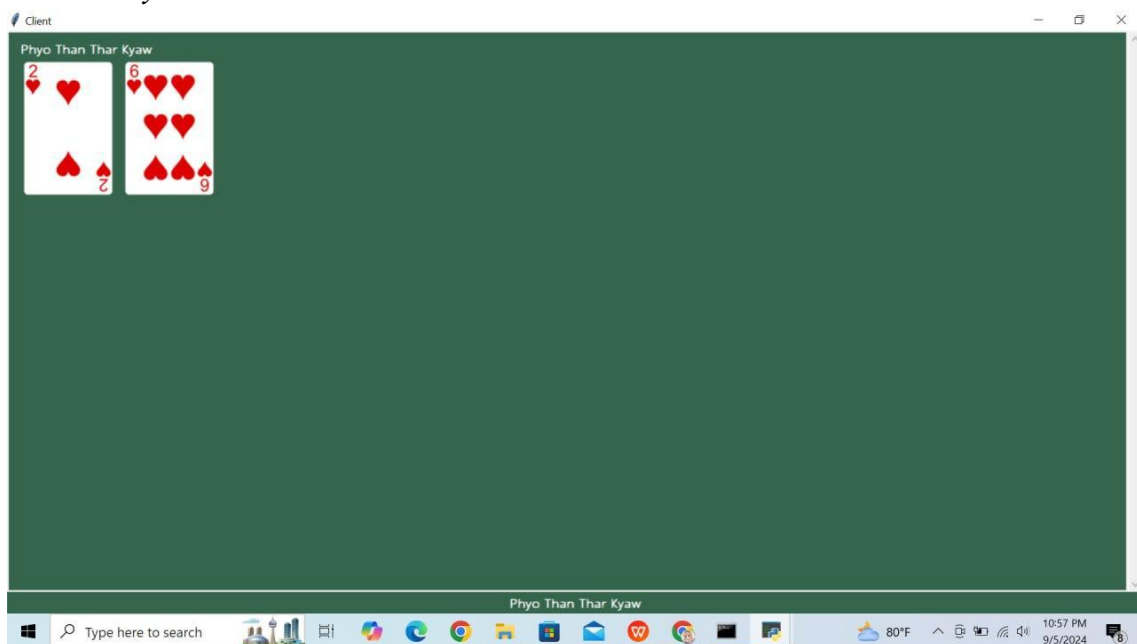


figure 3.12

Second Player's Screen

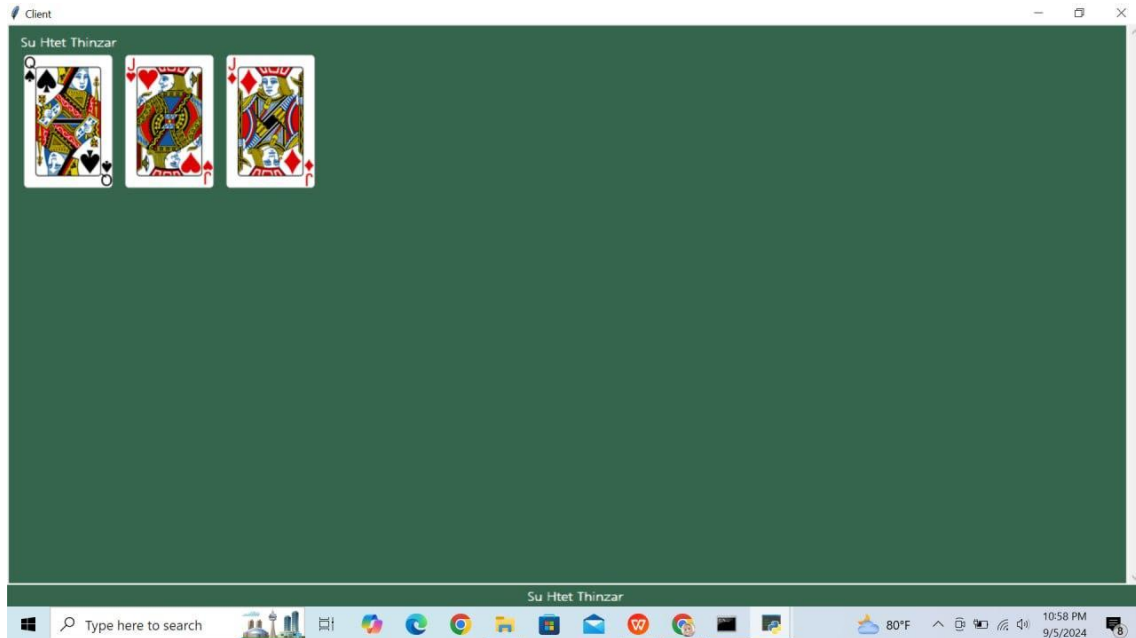


figure 3.13

This is the final game screen where the winner is shown. The cards of all the players will also appear as well. The name of the winner is written near the bottom of the screen.

Thiri Shwe Sin (First Player's Screen) showing the winner

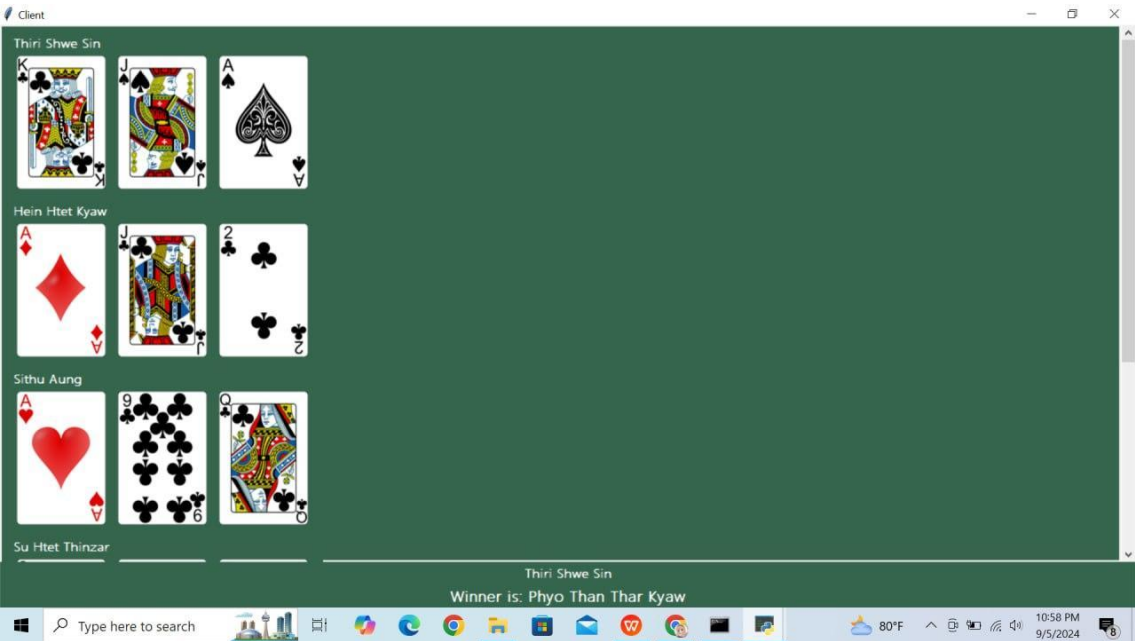
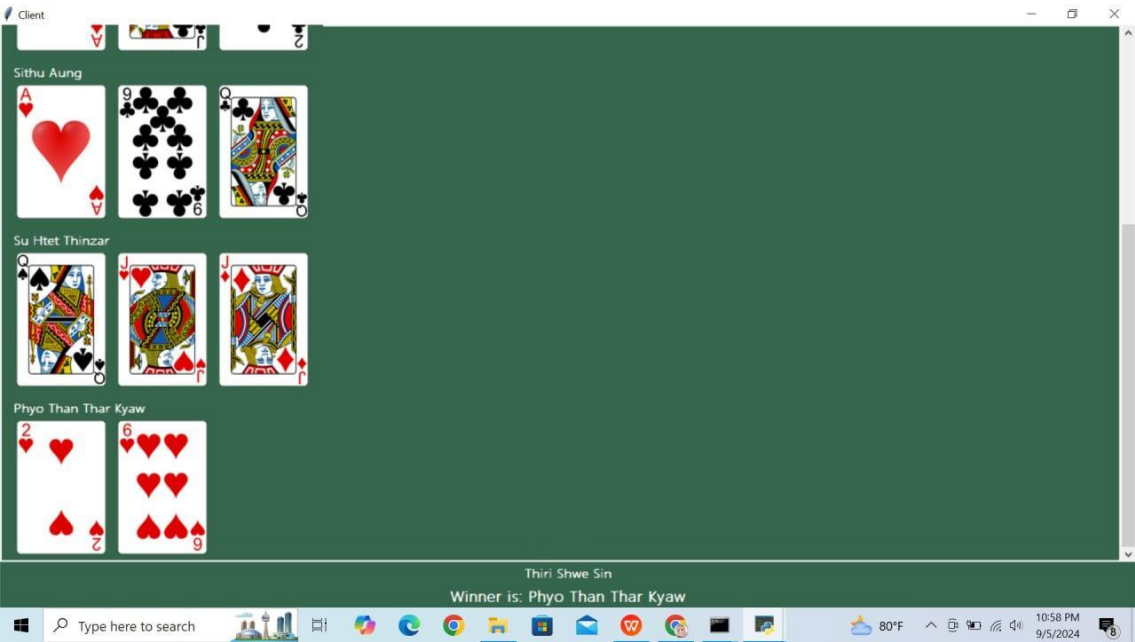


figure 3.14



4. Testing

4.1 Functionality testing

- Server Initialization: Verify that the server correctly initializes and accepts the number of players as input.
- Client Connection: Ensure that clients can successfully connect to the server after it's initialized.
- Account Validation: Test the login process to ensure that account credentials are validated correctly.
- Card Dealing: Check that the server correctly deals cards to each client.
- Winner Display: Ensure that the winner is correctly determined and displayed on both the client computers and the server.

4.2 Usability testing

- Interface Design: Evaluate the clarity of the GUI for both server and client..
- Navigation Flow: Test the flow from logging in to game setup and playing the game to ensure it is intuitive and user-friendly.
- Error Handling: Check how the game handles common errors (e.g., incorrect login details, lost connection) and how user-friendly the error messages are.

4.3 Reliability testing

- Consistency: Run multiple games in succession to verify that the game produces consistent results without crashing or errors.
- Data Integrity: Check that all data (e.g., player scores, account details) remains accurate and intact after several rounds of play.

4.4 Availability testing

- **Server Uptime:** Monitor the server's uptime during extended periods to ensure it remains available for clients to connect and play.
- **Client Accessibility:** Ensure that clients can connect to the server from different computers and networks without issues.
- **Concurrent Access:** Verify that multiple clients can join the game simultaneously and that the server can handle these connections without dropping any.

5. Application Configuration

5.1 Software Requirements

Server Side Configurations

- **Python Installation:** The server requires Python 3.x to be installed. Python is the primary programming language used for the game.
- **MySQL Connector:** The server needs the MySQL connector (mysql-connector-python) installed for database interactions, particularly for handling account validation and storing game results.
- **PIL (Python Imaging Library):** The server also requires PIL (or its modern equivalent, Pillow) for managing and displaying card images.
- **Required Libraries:** Other Python libraries used in the server-side script include socket for networking and tkinter for the GUI.
- **Database Setup:** The server must be configured to connect to a MySQL database, which stores user account information, game logs, and other relevant data.

Client Side Configurations

- **Python Installation:** Each client must have Python 3.x installed to run the game.
- **PIL (Python Imaging Library):** Similar to the server, the client requires PIL (or Pillow) for handling and displaying card images in the game interface.
- **Socket Library:** The client uses the socket library to connect to the server and communicate game actions.
- **Tkinter:** The client-side application also requires tkinter for the graphical user interface, which shows the cards and options to draw or pass.
- **Account Management:** The client interacts with the server to manage user accounts, which involves sending login credentials for validation.

5.2 Hardware Requirements

- The server side needs one computer and for clients it will need as many devices as input into the server.
- Any standard computer is enough to serve as the server or a client, provided it's properly configured with the necessary software and network access.

6. Conclusion

This Python-based multiplayer card game successfully integrates key programming concepts, including GUI design, database management, and real-time communication. The project delivers an engaging experience by combining Tkinter for an intuitive user interface, MySQL for reliable data storage, and Socket programming for smooth, real-time gameplay.

The game's unique scoring and tiebreaker rules, along with its client-server architecture, ensure a competitive and fair environment for players. The system supports future enhancements, such as AI opponents and performance optimization for larger player groups, making it a strong foundation for ongoing development.

Overall, the project demonstrates effective application of technical skills and the potential for further innovation in digital card games.

6.1. Limitations

Scalability Constraints:

The current system is designed to handle 2-5 players simultaneously. Scaling beyond this number may require performance optimizations and additional infrastructure to maintain smooth gameplay and responsiveness.

Network

Latency:

While Socket programming facilitates real-time communication, network latency can affect gameplay experience. Delays in data transmission may result in lag or synchronization issues, particularly in areas with unstable internet connections.

Limited

AI

Functionality:

The project does not include AI opponents, limiting gameplay to human players only. Future versions could benefit from the introduction of AI to offer single-player modes or practice against non-human competitors.

User**Interface****Constraints:**

The GUI, while functional, may lack advanced features or customization options. Enhancements to the interface could improve user experience, but these are not covered in the current implementation.

Database**Management:**

The MySQL database handles essential data storage but may face limitations in handling very large datasets or high concurrency. Optimizations may be required to support extensive player data and game history.

Error**Handling****and****Robustness:**

The system may not handle all edge cases or unexpected errors gracefully. Improvements in error handling and robustness are needed to ensure stability under diverse conditions.

6.2. Future Improvements

Looking ahead, several enhancements are planned to further enrich the card game project. First, the user interface, currently built with Tkinter, will be upgraded to offer a more engaging and visually appealing experience. Future updates aim to incorporate enhanced graphics, customizable themes, and more intuitive controls, which will improve overall user satisfaction.

Additionally, expanding the game library is a key goal. By introducing a variety of card games beyond the current implementation, such as Poker, Bridge, or Solitaire, the project will appeal to a broader audience and offer diverse gameplay experiences.

The incorporation of artificial intelligence (AI) is another significant improvement. Adding AI opponents will enable single-player modes and practice sessions against computer-controlled players, providing new gameplay options and increasing the game's versatility.

Performance optimization will also be a focus. To accommodate larger player groups and more complex scenarios, the system will need to improve network efficiency, optimize database queries, and scale server infrastructure to manage increased loads effectively.

Advanced error handling mechanisms will be implemented to ensure the system is more robust. Enhancements in error reporting and recovery processes will contribute to a more stable and reliable gaming experience.

Lastly, introducing customization and personalization features will allow players to tailor their game experience. Options for personalizing card designs, table layouts, and game rules through user settings will add a layer of engagement and personal touch to the gameplay.

These improvements are designed to enhance the game's functionality, appeal, and overall user experience.

7. References

- <https://www.datacamp.com/tutorial/a-complete-guide-to-socket-programming-in-python>
- <https://realpython.com/python-sockets/>
- <https://www.geeksforgeeks.org/socket-programming-python/>
- <https://docs.python.org/3/library/tkinter.html>
- <https://www.geeksforgeeks.org/python-gui-tkinter/>

